

An Alternative to FLOPS Regularization to Effectively Productionize SPLADE-Doc

Songwon Park

xlah1386@gs.cwnu.ac.kr

2025. 10. 30.

An Alternative to FLOPS Regularization to Effectively Productionize SPLADE-Doc

- **Aldo Porco and Dhruv Mehra and Igor Malioutov and Karthik Radhakrishnan and Moniba Keymanesh and Daniel Preotiuc-Pietro and Sean MacAvaney and Pengxiang Cheng**
(Bloomberg New York, United States)

- [paper](#)

메인 아이디어

학습된 희소 검색 모델에서 희소성을 유도하는 FLOPS에 문서 빈도(Document Frequency) 적용

- 학습된 희소 검색(Learned Sparse Retrieval, LSR) 모델
 - 가중치가 부여된 단어 벡터로 인코딩
 - 효율적으로 검색하기 위해서는 벡터가 희소해야 함
 - SPLADE의 경우 벡터의 희소성을 유도하기 위해 FLOPS 정규화를 이용
 - 특정 쿼리나 문서 내의 희소성만 보장할 뿐, 전체 쿼리 혹은 문서들의 단어간 희소성을 보장하지 않음
 - 문서 빈도(Document Frequency)가 높은 단어들은 검색 엔진에서 긴 포스팅 리스트를 생성해 검색 지연을 증가시킴
- ➔ 문서 빈도가 높은 단어의 사용을 억제하여 포스팅 리스트를 짧게 만들고 검색지연을 줄이는 DF-FLOPS 제안



- DF-FLOPS는 문서 빈도가 높은 단어의 사용을 효과적으로 줄이고 검색지연을 약 10배 감소시킴
- 도메인 내 성능의 경우 2.2% 감소
- 도메인 외 성능의 경우 13개 중 12개의 태스크에서 향상

Learned Sparse Retrieval

- 전통적인 bag-of-words 기반의 희소 벡터처럼 쿼리와 문서를 각 토큰의 중요도 점수로 표현
- 그러나 전통적인 희소 검색과 달리 LSR 벡터는 입력에 등장하지 않는 단어들도 포함
→ 이러한 단어의 확장으로 인해 벡터가 조밀해짐
- 희소성은 검색 효율성에 핵심적이기 때문에 희소성을 높이기 위한 다양한 방법이 제안됨
 - 학습 중 정규화 기반 희소화 : FLOPS, L1
 - 추론 중 희소화: Top-k pruning, Score Thresholding, 불용어(Stopword) 제거
- 모델상에서 효율을 높이려는 시도
 - SPLADE-Doc의 경우 쿼리 인코더를 제거하고, 쿼리를 단순 이진 토큰 집합으로 표현
- LSR 모델은 거의 모든 문서 벡터에 동일한 단어를 포함시키려는 경향이 있어, 문서 빈도가 높아지고 검색 지연이 커지는 문제 발생
 - 의미 확장을 하다보니 자주 나타나는 불용어에도 점수를 주는 경향 존재

SPLADE

- BERT의 Masked Language Model(MLM)의 출력을 이용해 입력 토큰에 대한 vocab의 토큰들의 중요도(활성화값)를 구함(w_{ij})

$$w_{ij} = \text{transform}(h_i)^T E_j + b_j$$

- w_{ij} : 입력 시퀀스의 토큰 i에 대한 vocab의 토큰 j의 중요도
- h_i : 입력 시퀀스의 토큰의 임베딩
- E_j : vocab 토큰 j의 임베딩
- b_j : 바이어스
- transform : GeLU 활성화 및 정규화를 이용하는 선형변환

- 최종 표현은 모든 입력 토큰에 대해 ReLU, log포화를 거친 후 가장 강하게 활성화 된 값 선택하여 구성

$$w_j = \max_{i \in t} \log(1 + \text{ReLU}(w_{ij}))$$

$$\text{Loss} = L_{\text{in-batch negative}} + \lambda_q \cdot L_{\text{flops-q}} + \lambda_d \cdot L_{\text{flops-d}}$$

학습 중 정규화 기반 희소화 - FLOPS 정규화

- FLOPS 정규화는 문서-쿼리 간 연산량이 줄어들도록 하는 역할
 - 평균적으로 활성화 확률이 큰 단어들을 억제하기 위해 벡터가 희소해지도록 유도
 - 활성화 확률이 높은 토큰의 경우 a_j 의 값이 다른 토큰보다 크기때문에 Loss값이 커짐
- 활성화 확률이 높은, 너무 자주 나타나는 단어들의 활성화 확률을 낮추기 위한 정규화

$$L_{FLOPS} = \sum_{j \in V} a_j^2 = \sum_{j \in V} \left(\frac{1}{B} \sum_{i=1}^N w_j^{(d_i)} \right)^2$$

- a_j : Batch 내의 문서들에 대해 토큰 j의 평균적인 활성화 확률
- B : Batch Size
- V : vocab
- $w_j^{(d_i)}$: 문서 d_i 에서 vocab의 토큰 j의 활성화 확률

추론 중 희소화 - Top-k pruning

- 임베딩 벡터에서 활성화 된 토큰들 중 상위 k개의 토큰만 남기고 나머지의 활성화 확률은 0으로 변환
- k가 작아질 수록 메모리 / 계산 지연이 줄어들지만, 과한 경우 매칭 손실이 일어날 수 있음.
- 본 논문에서는 k=150을 이용해 실험

추론 중 희소화 - score thresholding

- 활성화 확률들 중 임계값 이하의 토큰들은 모두 0으로 변환
- 의미가 약한(낮은 활성화 확률을 가진) 토큰들이 제거될 수 있으나 임계값 설정이 어렵고 태스크에 따라 최적 임계값이 크게 달라질 수 있음

추론 중 희소화 - Stopword removal

- 리스트를 기반으로 불용어를 전부 제거
- 문서 빈도가 높은 불용어들을 제거할 수 있지만, WHO(세계보건기구)와 같이 맥락상 중요한 토큰까지 제거될 위험이 존재

SPLADE-Doc

- SPLADE와 달리 문서만 인코딩하는 구조
 - 문서만 BERT의 MLM을 통과하여 각 vocab의 토큰 j 에 대한 활성화값을 계산
 - 쿼리의 경우 쿼리에 포함된 토큰 id만 이용
 - 유사도 계산시 쿼리에 나타난 토큰 id들에 대한 문서의 토큰 id들의 활성화 확률을 더해서 계산
- ex) 쿼리에 활성화된 토큰 id list = [15, 20, 36]
- 문서 1의 쿼리 토큰id에 대한 활성화 확률 : [..., 3.0, ..., 15.0, ..., 2.0, ...] $\rightarrow s(q, d) = 20.0$

$$s(q, d) = \sum_{j \in q} w_j^d$$

$$Loss = L_{in-batch \ negative} + \lambda_d \cdot L_{flops-d}$$

- 본 논문에서는 문서 빈도가 높은 용어에 가중치를 부여하여 LSR 모델의 자연 문제를 완화하는 방법 제안
- 추론 시점에서 DF를 줄이는 방식과 달리 학습 중에 DF-FLOPS 정규화를 적용하면 모델이 높은 문서 빈도 토큰에 과하게 의존하지 않으면서도 적절하게 불용어(WHO가 세계보건기구를 의미하는 경우) 사용 가능

Representation with FLOPS
Document: Estimates of disease burden and cost - effectiveness - WHO / C : Nelson : Disease burden is an indicator of health outcome - disease burden can be expressed in many ways ; such as the number of cases (e : g : incidence or prevalence) ; deaths or disability - adjusted life years lost (DALY s) associated with a given condition : Information on the reported incidence of vaccine - prevent able diseases is provided to WHO : Expansions: ('what', 6.18), ('is', 5.5), ('of', 5.1), ('the', 4.87), ('a', 4.83), ('who', 4.81), ('does', 4.66), ('in', 4.61), ('are', 4.4), ('an', 4.39), ('for', 4.19), ('how', 4.13), ('?', 4.02), ('disease', 3.89), ('burden', 3.72), ('diseases', 3.54), ('definition', 3.51), ('health', 3.49), ('and', 3.41), ('mean', 3.4)
Representation with DF-FLOPS
Document: Estimates of disease burden and cost - effectiveness - WHO / C : Nelson : Disease burden is an indicator of health outcome - disease burden can be expressed in many ways ; such as the number of cases (e : g : incidence or prevalence) ; deaths or disability - adjusted life years lost (DALY s) associated with a given condition : Information on the reported incidence of vaccine - prevent able diseases is provided to WHO : Expansions: ('burden', 3.59), ('disease', 3.44), ('estimates', 3.21), ('effectiveness', 3.12), ('diseases', 3.07), ('estimate', 2.98), ('indicator', 2.93), ('indicators', 2.82), ('nelson', 2.8), ('estimation', 2.67), ('load', 2.62), ('health', 2.55), ('vaccine', 2.53), ('illness', 2.52), ('DALY', 2.52), ('estimated', 2.51), ('who', 2.44), ('weight', 2.43), ('cost', 2.42), ('effective', 2.36)

➡ FLOPS는 의미적으로 무관한, 문서 빈도가 높은 “is”와 같은 토큰을 삭제하지 못하는 반면

DF-FLOPS는 “is”를 제거하고 의미적으로 중요한 “WHO”를 유지

Table 1: FLOPS and DF-FLOPS representations for a passage. ¹

문서 빈도에 조건화 된 FLOPS

- FLOPS 정규화는 벡터를 더 희소하게 유도하여(벡터에서 0이 아닌 원소 수를 줄여) 유사도 계산시 필요한 연산수를 감소시키는데 사용
- 높은 문서 빈도(DF)를 가진 용어로 인해 지연이 커지는 문제
 - 토큰 t 의 문서 빈도(DF)에 따라 가중치를 부여하는 DF-FLOPS 제안
 - 문서에 나타나는 빈도가 클수록 더 세게 정규화(FLOPS)가 걸리도록 제어

$$l_{DF-FLOPS} = \sum_{t \in V} \left(\frac{w_t}{B} \sum_{i \in 1}^N r_{i,t} \right)^2 \quad \text{where } w_t = \text{active}\left(\frac{DF_t}{|C|}\right)$$

- V : Vocab
- B : batch size
- $r_{i,t}$: i 번째 벡터(문서)에서 토큰 t 의 가중치
- DF_t : 토큰 t 의 문서 빈도에 대한 근사치
- $|C|$: DF를 근사할 때 사용한 코퍼스 크기
- active : 활성 함수

활성 함수와 튜닝

- $x = \frac{DF_t}{|C|}$ (토큰 t의 문서 빈도 비율) 가 커지면 가중치($w_t = \text{active}(\frac{DF_t}{|C|})$)가 커져 토큰에 더 강한

정규화가(FLOPS) 걸림

- $x < \alpha$ 이면 가중치가 상대적으로 약함
- $x = \alpha$ 이면 $\text{active}(x; \alpha, \beta) = \frac{1}{1 + (x^{\log_2 2} - 1)^\beta} = \frac{1}{1 + (2 - 1)^\beta} = \frac{1}{2}$
- $x > \alpha$ 이면 가중치가 상대적으로 빠르게 강해짐
- β 가 클 때 : x (토큰 t의 DF 비율)이 α 보다 살짝만 커져도 가중치가 급격히 커짐
- β 가 작을 때 : x (토큰 t의 DF 비율)이 α 보다 커진 이후 서서히 기증치가 커짐

$$\text{active}(x; \alpha, \beta) = \frac{1}{1 + (x^{\log_2 2} - 1)^\beta}$$

- $x = \frac{DF_t}{|C|}$
- α, β : 가중치가 어디서/얼마나 급하게 커질지 조정하는 파라미터
- α : 가중치가 걸리기 시작하는 DF 비율의 중앙점
실험에서는 0.1이용 (상위 10%의 DF부터 강하게 가중치를 주겠다)
- β : α 값 근처에서 가중치가 얼마나 급격하게 커지는지를 정함
실험에서는 10 이용
- active : 활성 함수

Dataset

- MS-MARCO passage ranking dataset을 이용해 학습
- MS-MARCO dev, TREC2019, TREC2020, BEIR을 이용하여 테스트

효과(성능) 측정

- nDCG@10 : top-10 내에서 관련도가 높을수록, 순위가 높을 수록 점수를 더 주는 지표
- MRR@10 : top-10 안에 관련 문서가 얼마나 높은 순위에 있는가를 측정하는 지표
- Recall@1000 : 전체 정답 문서 중 상위 1000개 안에 몇%가 들어있는지를 측정하는 지표

효율/희소성 측정

- Latency Avg / Latency P99(ms) : 쿼리당 검색시 걸리는 속도
- Matches Avg(백만단위) : 쿼리당 실제로 매칭되어 유사도계산/정렬에 이용된 문서 수의 평균
- Top@1 Token DF : 가장 자주 등장하는 토큰의 문서 빈도 비율(높을수록 많은 문서에 포함)
- Avg. Emb. Length : 활성화 확률이 0이 아닌 토큰의 평균 개수

ID	Model Name	MS-Marco		TREC 19		TREC 20		Latency	Latency	Matches	Top@1	Avg. Emb.
		MRR@10	R@1K	NDCG@10	R@1K	NDCG@10	R@1K	Avg (ms)	P99 (ms)	Avg (M)	Token DF	Length
1	BM25	18.4*	85.3*	56.5*	74.5*	47.9*	80.5*	68.9	241.3	0.952	20.6%	27.7
2	SPLADE-Doc w/ FLOPS	32.2	92.4	65.6	69.6	62.9	75.2	922.0	1945.6	8.628	95.8%	583.8
3	+ <i>Pruning@150</i>	32.0	92.1	64.6	68.9	61.8	74.0	792.1	1664.4	8.621	95.7%	147.6
4	+ $\uparrow \lambda = 0.1$	29.2	88.8	60.3	66.8	56.4	72.7	331.6	708.8	4.111	43.4%	87.6
5	+ $\uparrow \lambda = 1$	28.3	88.4	56.8	67.1	53.2	72.3	160.9	347.0	1.970	17.7%	33.0
6	SPLADE-Doc w/ DF-FLOPS	30.0	92.9	59.5	72.5	59.6	78.7	161.0	341.7	1.907	8.0%	301.6
7	+ <i>Pruning@150</i>	29.7	93.0	60.2	72.6	60.2	80.0	87.8	187.8	1.078	5.2%	140.3

Table 2: In-domain effectiveness results. On the left, model performance on MS-Marco dev set, TREC'19 and 20. On the right, latency related measures such as average number of matches per query, frequency of the most repeated token in the embedding, and average representation length. A * indicates that the result was copied from [8] (MS-Marco and TREC 19) or calculated using the PISA Python bindings (TREC 20) [15].

RQ1. FLOPS로 학습한 SPLADE-Doc의 지연을 유발하는 주된 요인은?

- 기존 FLOPS를 사용한 SPLADE-Doc의 경우 MS-MARCO에서 **MRR@10=32.2**로 높음
- 가장 자주 나타나는 토큰이 문서의 약 96%에 등장
 - 해당 토큰이 쿼리에 있다면 거의 전체 문서를 매칭/유사도계산/정렬 해야하기 때문에 지연 발생 (8.8M 개의 코퍼스 중 8.6M개의 코퍼스가 매칭됨)
 - ➔ **평균 지연이 BM25 대비 약 13배로 매우 높음**

ID	Model Name	MS-Marco		TREC 19		TREC 20		Latency	Latency	Matches	Top@1	Avg. Emb.
		MRR@10	R@1K	NDCG@10	R@1K	NDCG@10	R@1K	Avg (ms)	P99 (ms)	Avg (M)	Token DF	Length
1	BM25	18.4*	85.3*	56.5*	74.5*	47.9*	80.5*	68.9	241.3	0.952	20.6%	27.7
2	SPLADE-Doc w/ FLOPS	32.2	92.4	65.6	69.6	62.9	75.2	922.0	1945.6	8.628	95.8%	583.8
3	+ <i>Pruning@150</i>	32.0	92.1	64.6	68.9	61.8	74.0	792.1	1664.4	8.621	95.7%	147.6
4	+ $\uparrow \lambda = 0.1$	29.2	88.8	60.3	66.8	56.4	72.7	331.6	708.8	4.111	43.4%	87.6
5	+ $\uparrow \lambda = 1$	28.3	88.4	56.8	67.1	53.2	72.3	160.9	347.0	1.970	17.7%	33.0
6	SPLADE-Doc w/ DF-FLOPS	30.0	92.9	59.5	72.5	59.6	78.7	161.0	341.7	1.907	8.0%	301.6
7	+ <i>Pruning@150</i>	29.7	93.0	60.2	72.6	60.2	80.0	87.8	187.8	1.078	5.2%	140.3

Table 2: In-domain effectiveness results. On the left, model performance on MS-Marco dev set, TREC'19 and 20. On the right, latency related measures such as average number of matches per query, frequency of the most repeated token in the embedding, and average representation length. A * indicates that the result was copied from [8] (MS-Marco and TREC 19) or calculated using the PISA Python bindings (TREC 20) [15].

RQ2. Top-k Pruning으로 고빈도 토큰 문제를 해결할 수 있나?

- 직관적으로는 가능하나, 실제로는 고빈도 토큰의 가중치가 높게 남아있음
- pruning@k로도 충분히 제거되지 않음
 - ➡ 지연이 다소 줄더라도(922.0 → 792.1) 여전히 높음

ID	Model Name	MS-Marco		TREC 19		TREC 20		Latency	Latency	Matches	Top@1	Avg. Emb.
		MRR@10	R@1K	NDCG@10	R@1K	NDCG@10	R@1K	Avg (ms)	P99 (ms)	Avg (M)	Token DF	Length
1	BM25	18.4*	85.3*	56.5*	74.5*	47.9*	80.5*	68.9	241.3	0.952	20.6%	27.7
2	SPLADE-Doc w/ FLOPS	32.2	92.4	65.6	69.6	62.9	75.2	922.0	1945.6	8.628	95.8%	583.8
3	+ <i>Pruning@150</i>	32.0	92.1	64.6	68.9	61.8	74.0	792.1	1664.4	8.621	95.7%	147.6
4	+ $\uparrow \lambda = 0.1$	29.2	88.8	60.3	66.8	56.4	72.7	331.6	708.8	4.111	43.4%	87.6
5	+ $\uparrow \lambda = 1$	28.3	88.4	56.8	67.1	53.2	72.3	160.9	347.0	1.970	17.7%	33.0
6	SPLADE-Doc w/ DF-FLOPS	30.0	92.9	59.5	72.5	59.6	78.7	161.0	341.7	1.907	8.0%	301.6
7	+ <i>Pruning@150</i>	29.7	93.0	60.2	72.6	60.2	80.0	87.8	187.8	1.078	5.2%	140.3

Table 2: In-domain effectiveness results. On the left, model performance on MS-Marco dev set, TREC'19 and 20. On the right, latency related measures such as average number of matches per query, frequency of the most repeated token in the embedding, and average representation length. A * indicates that the result was copied from [8] (MS-Marco and TREC 19) or calculated using the PISA Python bindings (TREC 20) [15].

RQ3. FLOPS의 계수(λ)를 더 키워서 해결이 가능한가?

- 계수 를 키우면 표현이 더 희소해짐
 - 평균 활성화 되는 토큰 수 감소, 최빈 토큰 약 44% → 약 18%
- 위의 변화로 지연 시간이 단축되지만 성능이 저하된다는 문제 발생

ID	Model Name	MS-Marco		TREC 19		TREC 20		Latency	Latency	Matches	Top@1	Avg. Emb.
		MRR@10	R@1K	NDCG@10	R@1K	NDCG@10	R@1K	Avg (ms)	P99 (ms)	Avg (M)	Token DF	Length
1	BM25	18.4*	85.3*	56.5*	74.5*	47.9*	80.5*	68.9	241.3	0.952	20.6%	27.7
2	SPLADE-Doc w/ FLOPS	32.2	92.4	65.6	69.6	62.9	75.2	922.0	1945.6	8.628	95.8%	583.8
3	+ <i>Pruning@150</i>	32.0	92.1	64.6	68.9	61.8	74.0	792.1	1664.4	8.621	95.7%	147.6
4	+ $\uparrow \lambda = 0.1$	29.2	88.8	60.3	66.8	56.4	72.7	331.6	708.8	4.111	43.4%	87.6
5	+ $\uparrow \lambda = 1$	28.3	88.4	56.8	67.1	53.2	72.3	160.9	347.0	1.970	17.7%	33.0
6	SPLADE-Doc w/ DF-FLOPS	30.0	92.9	59.5	72.5	59.6	78.7	161.0	341.7	1.907	8.0%	301.6
7	+ <i>Pruning@150</i>	29.7	93.0	60.2	72.6	60.2	80.0	87.8	187.8	1.078	5.2%	140.3

Table 2: In-domain effectiveness results. On the left, model performance on MS-Marco dev set, TREC'19 and 20. On the right, latency related measures such as average number of matches per query, frequency of the most repeated token in the embedding, and average representation length. A * indicates that the result was copied from [8] (MS-Marco and TREC 19) or calculated using the PISA Python bindings (TREC 20) [15].

RQ4. DF-FLOPS는 성능을 크게 떨어뜨리지 않으면서 희소성을 높일 수 있을까?

- DF-FLOPS는 높은 계수의 기존 FLOPS와 유사한 지연 시간을 얻으면서도 MRR@10에서는 +1.7, R@1000 + 4.5 성능 달성
- DF-FLOPS에 pruning@150을 결합하면 평균 지연이 거의 절반으로 줄어들지만 성능 저하는 미미함
- DF-FLOPS는 최빈 토큰을 8%까지 낮추지만, 평균 활성 토큰수는 높음
 - 모델이 고빈도 DF 토큰 대신 의미적으로 더 관련성 높은 저빈도 토큰을 쓰도록 유도되기 때문으로 해석됨

BEIR Dataset	BM25	FLOPS				DF-FLOPS	
		Base	Top150	$\lambda=.1$	$\lambda=1$	Base	Top150
arguana	31.5*	11.16	19.06	32.61	28.83	33.25	39.74
climate-fever	21.3*	6.89	9.15	12.49	11.96	13.44	13.58
dbpedia-entity	27.3*	31.21	30.83	30.31	30.55	32.73	32.26
fever	75.3*	57.67	50.47	58.22	60.49	63.12	60.67
fiqa	23.6*	19.64	19.28	21.29	21.08	25.56	24.86
hotpotqa	60.3*	42.08	43.89	48.9	48.59	55.34	55.85
nfcopus	32.5*	29.74	28.35	30.91	30.09	30.59	30.31
nq	32.9*	39.82	37.89	35.02	34.24	39.05	38.1
quora	78.9*	7.56	8.13	17.14	12.39	48.1	48.35
scidocs	15.8*	12.67	12.01	12.87	13.49	13.79	13.52
scifact	66.5*	58.75	55.36	63.4	60.87	65.6	64.25
trec-covid	65.6*	56.17	50.15	54.65	51.78	57.52	58.56
webis-touche2020	36.7*	23.28	23.38	21.79	21.45	25.97	24.56

Table 3: Out-of-domain performance (NDCG@10) on BEIR datasets.

A * indicates that the result was copied from [8]

RQ5. 도메인 외 일반화(BEIR)에는 어떤 영향이 있는가?

- DF-FLOPS로 학습한 SPLADE-Doc은 13개 중 12개의 BEIR 데이터셋에서 기존 FLOPS 대비 더 나은 성능을 보임
- 전반적으로 고빈도 토큰 과의존은 과적합을 유발할 수 있음

- FLOPS로 학습한 SPLADE-Doc에서 고빈도 토큰이 높은 가중치를 받아 벡터에 남아있어 검색 지연이 커지는 문제 존재
- FLOPS 손실만으로는 성능 저하 없이 해결하기 어렵다는 점 확인
- 문서 빈도를 이용한 DF-FLOPS를 제안
 - 문서 빈도가 높은 토큰에 가중치를 부여해 의미가 중요한 경우만 쓰이도록 함
- 기존 FLOPS를 사용한 SPLADE-Doc 지연 시간 대비 10배 개선
- MRR 성능 소폭 감소, recall@1000 성능 유지
- 향후 더 강력한 SPLADE 변형들에 접목하는 연구 가능



Making the Great! Making the New

감사합니다